kv and pred

Abstract

kv is a read-optimized key/value set with a hash signature that answers subset queries without touching the elements. pred builds on it: a label-selector style predicate that is compiled once and matched many times. Both trade construction cost for match cost, on the assumption that a set or a predicate is built once and queried repeatedly. They live at github.com/bowei/kvec/pkg/kv and github.com/bowei/kvec/pkg/pred, over the signatures in .../pkg/sig and the hash in .../pkg/fnv.

1 kv

1.1 Public interface

`KV` An immutable set of unique key/value pairs.

`New(itr func(yield func(k, v string) bool), cap int) *KV` builds a `KV` from an iterator over pairs. Keys are assumed unique; the constructor does not deduplicate. `cap` is a capacity hint equal to the expected number of pairs.

`(*KV) Len() int` the number of pairs.

`(*KV) Equal(other *KV) bool` whether both sides hold exactly the same pairs.

`(*KV) Contains(other *KV) bool` whether every pair of `other` is also in the receiver, i.e. whether `other` is a subset.

`(*KV) ContainsKeys(keys ...string) bool` whether the receiver holds a pair for every named key, whatever the value. No keys is vacuously true; duplicate keys are checked more than once.

`(*KV) Get(key string) (string, bool)` the value for a key, and whether such a pair exists.

`sig.Signature` A standalone, dynamically sized approximation of set containment over plain strings. It lives in `pkg/sig` alongside the compile-time sized vectors described below, one of which `KV` embeds; `kv` depends on that package rather than the other way round.

`NewSignature(bitWidth uint, keys ...string) *Signature` rounds `bitWidth` up to a power of two, minimum 64, and sets one bit per key. Duplicate keys are harmless. Panics above 2^{24} bits.

`(*Signature) BitWidth() uint` the width actually allocated.

`(*Signature) Contains(other *Signature) bool` whether every bit of `other` is set in the receiver. Panics on mismatched widths, since no answer would preserve the one-sided guarantee below.

`(*Signature) Equal(other *Signature) bool` same width, same bits.

`sig.Fixed64`, `sig.Fixed128`, `sig.Fixed256` The compile-time sized counterparts, of 64, 128 and 256 bits respectively. The width is in the type name, so the loops are unrolled and the vector sits inline in the caller's struct instead of behind a slice header. The three share one interface, with `N` standing for the width:

`(*FixedN) Set(bit uint)` sets the bit at an offset. Offsets at or above `FixedNBits` are dropped silently, so callers fold their hashes with `FixedNBits` − 1.

`(*FixedN) Contains(other FixedN) bool` whether every bit set in `other` is also set in the receiver.

`(*FixedN) Equal(other FixedN) bool` the same bits set.

The zero value is the empty signature, which every vector of that width contains. Unlike `Signature`, whose mismatched widths panic at the call, the widths here are distinct types that never meet: the choice is made once, where the field is declared, and the compiler holds it.

The guarantee is one-sided, for the fixed vectors and `Signature` alike. For string sets A, B with signatures S_A, S_B:

$$A \supseteq B \implies S_A.\text{Contains}(S_B),$$

but the converse does not hold, as distinct keys can fold onto the same bit. A signature therefore rejects with certainty and accepts only provisionally.

1.2 Internal optimizations

Fixed-width signatures as a rejection gate. Every `KV` carries two `sig.Fixed128` vectors of `sig.Fixed128Bits` = 128 bits, built at construction: `ksig` over the key hashes, and `kvsig` over the key hashes mixed with the value hashes. `Contains` tests `other`'s bits against the receiver's with `&^`; any bit present in `other` and absent in the receiver proves a pair the receiver cannot hold, and rejects before a single element is read. Both vectors are checked. This is not redundant: the two fold different hashes into the same width, so a pair missing from one can collide in the other, and at two words apiece both vectors sit in the cache line already fetched. The fixed width lets the loops be unrolled and keeps the vector inline in the struct, where the dynamic `sig.Signature` would cost an allocation and an unrollable loop.

Hash-ordered storage. Pairs are sorted once at construction by `kvCmp`, which orders on the 64-bit FNV-1a hash of the key and breaks ties on the key itself so the order is total. Hash order makes the sort key an integer comparison in the common case, and the totality is what `Equal`, `Contains`, `ContainsKeys` and `Get` all rely on for their merge and binary searches.

Galloping merge in `Contains`. Both sides being sorted, the subset test is a single merge pass. A pure merge costs a comparison per element of the larger side, which is wasteful when the receiver is much larger than `other`. The merge therefore counts consecutive skipped elements, and once the run reaches `gallopAfter` (= 4) it switches to a binary search over the remaining tail. The threshold is the point at which the walk has already cost about what $\log_2$ comparisons would, so the search never loses by much and wins outright on longer runs. The check sits inside the skip branch rather than at the top of the loop, so a merge of two similar sets — which never builds a run that long — does not pay for it. The pass also aborts early whenever more elements remain in `other` than in the receiver.

Per-probe signature gate. `ContainsKeys` and `Get` build a one-bit signature for the probe key and test it against `ksig` before searching. A missing key is usually rejected on a bit operation instead of a binary search.

Allocation-free hashing. `fnv.Hash64`, in `github.com/bowei/kvec/pkg/fnv`, folds bytes into a plain `uint64` rather than going through `hash.Hash`, on a path taken once per key at construction and once per probe at query time. Both `kv` and `sig` hash through it. The hash-object allocation this is meant to avoid does not in fact appear at a call site simple enough for the compiler to inline `New64a` and keep the object on the stack, and `BenchmarkStdlibHash64` measures the two within noise of each other on a twelve-byte key. What the package buys is a function returning a `uint64` that can be resumed with `Add64` and `AddWord64`, which is what `KV.Hash` folds its pairs with.

Power-of-two widths. Every signature type has a power-of-two bit width, so folding a hash to a bit offset is a mask (`hash & bitMask`) rather than a division. `Signature.Contains` reslices the other side to the receiver's length so the bounds-check eliminator can drop the checks in the loop body.

2 pred

2.1 Public interface

`PredicateBuilder` accumulates terms; each method returns the receiver so calls chain.

`NewPredicateBuilder() *PredicateBuilder` an empty builder.

`Key(keys ...string)` each key must be present, whatever its value.

`NoKey(keys ...string)` none of the keys may be present.

`Has(kv map[string]string)` each key must be present with the given value. A later call overwrites an earlier one on the same key.

`ValueIn(key string, values []string)` the key must be present with a value drawn from `values`. An empty `values` makes the predicate unsatisfiable.

`ValueNotIn(key string, values []string)` the key must be absent, or present with a value outside `values`.

`Build() *Predicate` compiles the accumulated terms. The result is independent of the builder, and `Build` may be called more than once.

`Predicate` is the compiled, immutable conjunction of those terms. `(*Predicate) Match(kv *kv.KV) bool` reports whether a `KV` satisfies every term; a predicate built from an empty builder matches everything. Being immutable, one `Predicate` can be matched against many `KV`s from many goroutines.

```
p := NewPredicateBuilder().
    Key("app").
    NoKey("deprecated").
    Has(map[string]string{"tier": "backend"}).
    ValueIn("env", []string{"prod", "staging"}).
    Build()
```

2.2 Internal optimizations

The builder stores terms as given and does no analysis. All of it happens once, in `Build`, so that `Match` does the least possible work per candidate.

Merging repeated terms. Repeated `ValueIn` terms on one key are intersected and repeated `ValueNotIn` terms are unioned, so a key costs one lookup at match time however many times it was named.

Subsumption. A term that settles a key's value makes weaker terms on that key dead, and they are dropped:

- an equality decides both value tests on its key at build time, and makes a `Key` on it redundant;
- an in-set absorbs an exclusion on the same key by subtracting it, and itself proves the key present, so a `Key` on it is also dropped;
- a `NoKey` satisfies an exclusion on the same key vacuously, so the exclusion is dropped.

Demotion to equality. An in-set reduced to a single value is rewritten as an equality. That moves it behind the signature gate of `kv.KV.Contains`, turning a lookup into part of a bit test.

Batched equalities. All pinned pairs are compiled into one `kv.KV`, so the whole group is tested by a single `Contains` rather than a lookup per pair — and rejected on two words of signature in the common case.

Contradictions resolved once. Conflicts between terms (an equality outside its in-set, an in-set emptied by subtraction, a key required both present and absent) are found at build time and recorded in a single `unsatisfiable` flag, so `Match` returns on one boolean.

Cheapest-first evaluation. `Match` orders its terms by cost: the equality set first, since its signature gate rejects the whole group without a lookup and it is the only test that can reject on a wrong value rather than a missing key; then the existence tests, each usually rejecting on a signature bit alone; then the value tests, which always pay for a lookup.

`sortedset`. Value sets are sorted and deduplicated at build time, which both shrinks them and enables searching. `contains` scans linearly up to `linearScanMax` (= 8) elements, where locality and branch prediction beat the $\log_2 n$ comparisons of a binary search, and binary searches above it. `intersect`, `union` and `subtract` are single linear merges over the sorted representation.

Deterministic layout. The per-key sets are flattened into slices ordered by key, so a compiled `Predicate` does not depend on map iteration order and matches in a stable, cache-friendly sequence.

3 Benchmark results

All numbers below were measured on an Apple M3 under `go1.25.1 darwin/arm64` with `go test -bench . -benchmem -count 5`; each figure is the median of the five runs, in nanoseconds per operation. Run-to-run spread was at most 5% of the median, and under 1% for most figures. Every operation on the query path allocates nothing: only `Build` appears with a non-zero allocation count.

3.1 `kv.KV`

Table 1 is a 64-pair set. The pattern the design is built around shows up directly: a miss is more than an order of magnitude cheaper than a hit, because the signature answers it without reaching the elements.

Operation	ns/op	Path taken
`Contains` hit (2 pairs)	48.6	merge, then compare values
`Contains` miss (2 pairs)	1.4	rejected on `kvsig`
`ContainsKeys` hit	32.3	signature passes, binary search
`ContainsKeys` miss	6.0	rejected on `ksig`
`ContainsKeys` 4 keys, all hit	129.4	four searches
`ContainsKeys` 5 keys, last misses	133.1	four searches, then reject

Table 1: `KV` operations against a 64-pair set.

3.2 `Contains` across sizes

Table 2 times `Contains` for every pair of set sizes. At these sizes both signatures are saturated on both sides, so they never reject: this is the case where the merge has to do the work.

	`other`			
receiver	1k	10k	100k	1M
1k	4 667	1.7	1.8	1.8
10k	6 451	47 745	1.7	1.7
100k	8 180	101 523	470 658	1.8
1M	16 218	188 793	2 907 257	4 704 127

Table 2: `Contains` in ns/op. Below the diagonal the call is a genuine hit that runs the merge to completion; above it the receiver is the smaller set and the call exits on the length check.

Two things are worth reading off it. First, the upper triangle is flat at 1.7–1.8 ns: a set that cannot possibly be a superset is rejected in constant time, whatever the sizes involved. Second, the

lower triangle is where the gallop earns its keep. Probing 1k pairs into a 1M-pair set costs 16.2 μs, about 16 ns per probe — the cost of a binary search each. Walking that set instead would cost roughly what the 1M/1M cell costs, 4.7 ms, so on this shape the gallop is around 290× ahead. The diagonal, where the two sides are the same size and the gallop never triggers, runs at about 4.7 ns per pair.

3.3 `sig.Signature`

	64 bits	1024 bits	65536 bits
hit	1.9	6.4	283.8
miss	1.4	2.9	92.4

Table 3: `Signature.Contains` in ns/op, 64 keys in the receiver.

Cost is linear in the number of words, since a hit has to read all of them; a miss is cheaper because the loop returns at the first word that fails. This is the argument for the inline `sig.Fixed128` inside `KV`: at two words the whole test is the 1.4–1.9 ns line of Table 3, with no allocation and no loop.

3.4 Where the signature gate stops paying

The gate is only worth what it rejects, and what it rejects decays as the set fills it. A `KV` carries `sig.Fixed128Bits` = 128 bits, so a set of n keys leaves a fraction of about

$$\left(1 - \frac{1}{128}\right)^n$$

of them clear, and only an absent key landing on a clear bit is rejected without a search. Table 4 measures both the rate and its cost. The share is counted over 4096 absent keys; the timing rotates through 64 of them, so each figure averages over the gate firing and not firing rather than depending on whether one chosen key happens to collide.

pairs in the set	8	16	32	64	128	256	1024
absent keys rejected by `ksig`	92.9%	87.2%	75.3%	64.5%	31.2%	13.9%	0.0%
predicted by the formula	93.9%	88.2%	77.8%	60.5%	36.6%	13.4%	0.0%
`ContainsKeys` miss, ns/op	12.8	13.5	14.7	16.4	30.5	43.3	50.6

Table 4: Decay of the fixed signature as the set grows. Measured rates track the formula to within a few points.

The cost of a missing key rises from 12.8 to 50.6 ns as the gate goes from rejecting almost everything to rejecting nothing, at which point every probe pays for the binary search the signature exists to avoid. This is the quantitative form of the note in the source that `sig.Fixed128` suits sets of far fewer than a thousand pairs: the gate is worth having up to a hundred or so pairs, is half wasted by 128, and is dead weight by 1024.

Note that the single figure in Table 1 — a 6.0 ns `ContainsKeys` miss at 64 pairs — is one key that happened to land on a clear bit. Table 4 gives the honest average for that size, 16.4 ns.

3.5 Choosing the width

`sig.Fixed64` and `sig.Fixed256` are the same structure at half and twice the bits, costing one word less and two words more per vector. The formula above depends on n and the width only through

their ratio, so doubling the width shifts the whole decay curve one column to the right of Table 4, whose columns are powers of two.

pairs in the set	8	16	32	64	128	256	1024
`Fixed64`	88.2%	77.7%	60.4%	36.5%	13.3%	1.8%	0.0%
`Fixed128`	93.9%	88.2%	77.8%	60.5%	36.6%	13.4%	0.0%
`Fixed256`	96.9%	93.9%	88.2%	77.8%	60.6%	36.7%	1.8%

Table 5: Predicted share of absent keys the gate rejects, by width. Each row is the one above it shifted by a column: a doubling of the width buys a doubling of the set size at which the gate still pays.

Table 5 is predicted, not measured. Only the `Fixed128` row has been measured — it is the formula row of Table 4, which tracked the observed rate to within a few points — and `KV` still embeds that width, so every figure elsewhere in this section stands unchanged. What the other two rows say is where the choice moves the cliff, not what it costs.

The cost side is the other half of it. A wider vector makes the gate itself slower, since a hit has to read every word, and Table 3 puts that growth at roughly linear in words. Four words is still a handful of instructions over data already inline in the struct, and it is bounded above by the dynamic `Signature` figures there, which pay for a slice header and an unrollable loop on top. The trade is therefore a few extra instructions on every gate against a binary search avoided on the misses the narrower vector would have let through, which is worth taking only where the sets really do run large. A `KV` known to hold a handful of pairs can go the other way and take `Fixed64`, keeping most of the gate — 88.2% at eight pairs against 93.9% — for one word instead of two.

3.6 `pred`

`Match` checks its terms cheapest-first, so a single "miss" figure would say nothing: it would report whichever term the input happened to break. Table 6 instead breaks a different term in each column, in the order `Match` tries them.

rejected on		`KV` size		
term	broken by	8	16	64
`eq`	key absent	2.4	2.4	2.4
`eq`	wrong value	2.4	2.4	2.4
`keyExists`	required key gone	45.2	66.7	77.9
`keyNExists`	forbidden key present	93.8	104.1	122.5
`in`	value outside the set	91.2	103.9	120.9
`notIn`	value inside the set	116.4	132.7	155.6
accepted (all terms pass)		117.1	133.3	157.2
unsatisfiable predicate			1.9	

Table 6: `Match` in ns/op, for a five-term predicate naming one term of every kind.

The spread is the result. A candidate rejected on the equality set costs 2.4 ns whatever its size, against 157.2 ns for one that passes every term — a factor of 65 between the cheapest and dearest outcome of the same call. Ordering the terms is worth that factor only because the cheap tests are also the ones most likely to fire: a selector applied across a population rejects most of it, and rejects most of that on a pinned pair.

Three details are worth reading carefully.

- A wrong value costs exactly what an absent key costs, 2.4 ns. The `eq` test is not a lookup that happens to fail early — `kvsig` folds the value into the bit, so a pair with the right key and the wrong value misses the signature just as a missing key does.
- The rows are not a strict ladder, and cannot be: breaking a term also changes what the earlier terms do. Rejecting on `keyNExists` requires the forbidden key to be *present*, which turns the lookup that rejects into a hit rather than a signature miss, and that is why it costs about what the later `in` row costs. Each row is the true cost of that outcome, but the rows are not a decomposition of one call.
- Everything on this path is allocation-free at every size.

`Build` costs 953.9 ns and 1584 B in 30 allocations for this predicate. That is the bet of the package as a number: against the 157.2 ns accept, it pays for itself after about six matches, and against the 2.4 ns reject, after roughly four hundred. Either way a predicate matched across a population of any size has long since repaid it.

The same five terms written so that two of them are redundant — a `Key` and a `ValueNotIn` on a key that an equality already settles — build in 837.0 ns and 25 allocations, and compile to three terms rather than five. The redundant form is both cheaper to build and cheaper to match, since the work it names is discarded rather than performed once per candidate.

3.7 Why the equality set is one `Contains`

`BenchmarkEqStrategy` compares the two ways of testing three pinned pairs: the single `Contains` that `Match` uses, against the lookup-per-pair it replaced.

	`KV` size	8	16	64
hit	`Contains`	41.8	47.2	55.3
	per-pair lookup	99.0	109.0	130.6
miss	`Contains`	1.9	1.9	1.9
	per-pair lookup	83.9	92.6	109.0

Table 7: Testing three pinned pairs, in ns/op.

`Contains` is 2.3–2.4× faster on a hit, where it makes one merge pass instead of three independent searches. On a miss it is 45–58× faster, and — more to the point — flat at 1.9 ns across every size, where the lookup form grows with the candidate. That flat line is the 2.4 ns `eq` row of Table 6 seen on its own, and it is the reason `Build` folds every equality into a single `kv.KV` and `Match` tests it first.